



Proyecto I: LRU Caché

Algoritmos y Estructuras de Datos — 2026-2
27 de septiembre de 2026

Grupo 3 - Teoría 3

Nombre completo	Código del estudiante
Abigail Jaslin Cabanillas Ventocilla	202510438
Azul Arbulú Silva	202510303
Kiara Luz Rojas Meza	202510620

Docente: Victor Racso Galvan Oyola.

1. Descripción de la estructura y *TDA*

El *LRU Cache* utiliza una estructura híbrida que combina una *Tabla Hash* y una Lista Doblemente Enlazada. Por un lado, la *Tabla Hash* permite realizar la búsqueda de cualquier elemento en tiempo $O(1)$, y almacena como valor la dirección en memoria; en la Lista Doblemente Enlazada, del elemento que está como clave. Esta última estructura, mantiene todos los elementos ordenados estrictamente por su antigüedad de uso. De esta forma, el *LRU Cache* puede acceder al primer y último elemento, o mover cualquier elemento intermedio hacia el frente, en tiempo constante $O(1)$.

Asimismo, el *LRU Cache* es un tipo de dato abstracto que define una estructura de almacenamiento con una capacidad máxima fija. Su comportamiento es el siguiente: cuando la caché alcanza su límite de capacidad y se quiere insertar un nuevo elemento, el *TDA* identifica y elimina el elemento menos usado recientemente. Consta de tres métodos clave: `init(capacidad)` para establecer su capacidad, `get(clave)` para recuperar un valor y marcarlo como el más reciente, y `put(clave, valor)` para insertar o actualizar elementos.

2. Herramientas usadas y pasos para reproducir el video

2.1. Herramientas

- *C++17*, implementación real de la *LRU Cache* (*hash map* + lista doblemente enlazada) y generación de los eventos de traza.
- *ManimCE*, animación en *Python*, impulsada por los trazes reales generados por *c++*.
- *CMake*, build del proyecto *C++* (*targets main_A* y *test_lru*).
- *ffmpeg*, usado internamente por *Manim* para el render final a *.mp4*.
- Instalar *Manim*: `pip install manim` (requiere *pycairo/pangocairo* del sistema: `apt install libcairo2-dev libpango1.0-dev pkg-config`).
- Renderizar cada escena: `manim -pql scenes.py NombreDeLaEscena` para pruebas rápidas, `-pqh` para la calidad final.

2.2. Pasos a seguir

1. Clonar el repositorio https://github.com/Kroj-07/LRU-AED-26_2.git.
2. Compilar y correr las pruebas para generar los *traces* reales: `g++ -std=c++17 -O2 test_lru.cpp -o test_lru && ./test_lru`

3. Contribuciones de cada integrante

Integrante	Contribución de código	Contribución a la animación	Contribución al informe y/o README.md
Abigail Jaslin Cabanillas Ventocilla	Casos de prueba y <i>script</i> principal para ejecutar todo el flujo.	Animación de la introducción, <i>TDA</i> y utilidad.	Creación y borrador del README.md. Conclusiones y evidencias para el informe.
Azul Arbulú Silva	Código del caso de caché llena o vacía de <i>eviction</i> , generación del <i>trace.json</i> .	Animación del caso de caché llena o vacía de <i>eviction</i> .	Sección de análisis de complejidad para el informe como actualizar el README.md y pasar informe a <i>LaTeX</i> .
Kiara Luz Rojas Meza	Creación de la clase <i>LRUCache</i> , implementación de los métodos <code>get()</code> / <code>put()</code> .	Animación de los métodos <code>get()</code> / <code>put()</code> .	Secciones de descripción, estructura y Tipo de Dato Abstracto (<i>TDA</i>). para el informe.

4. Análisis de complejidad

Operación	Complejidad	Explicación breve
<code>get(key)</code>	$O(1)$	<code>moveToFront</code> reasigna punteros y encontrar el nodo tiene una complejidad de $O(1)$.
<code>put(key, value)</code> (si existe)	$O(1)$	La búsqueda como la actualización del valor y el <code>moveToFront()</code> toman $O(1)$.
<code>put(key, value)</code> (nuevo)	$O(1)$	La inserción como el <code>insertFront()</code> toman $O(1)$ aunque hubiese <i>eviction</i> .
<code>evict()</code>	$O(1)$	El puntero anterior a la cola da el LRU directamente, <code>removeNode()</code> reasigna punteros y <code>map.erase</code> toma $O(1)$.

5. Repositorio de *GitHub*

https://github.com/Kroj-07/LRU-AED-26_2.git

6. Conclusiones

- La eficiencia $O(1)$ de la *LRU Cache* no surge de una sola estructura, sino de la complementariedad entre una tabla *has* (acceso directo por clave) y una lista doblemente enlazada (orden de uso mantenido explícitamente); ninguna de las dos por separado resuelve el problema en tiempo constante.
- Separar la lógica real (*C++*) de la capa de animación (*Manim*, vía un tarce de eventos en *JSON Lines*) permitió cumplir el requisito de “animación real, no simulada” sin acoplar el motor gráfico a lenguajes de la estructura, y facilitó verificar cada caso desborde (caché vacía, capacidad 1, *eviction*) de forma independiente antes de animar